

Transformer Debugger — Full Project Plan

An interactive mechanistic interpretability tool: think "VS Code debugger, but for transformer internals." Built as a Python package on top of TransformerLens, giving researchers a GUI to explore, visualize, and run causal experiments on any decoder-only LLM — instead of writing notebook code from scratch every time.

1. The Core Idea

Load any transformer model and interactively inspect what's happening inside it: residual stream, attention heads, neuron activations, SAE features, logit predictions at each layer, and causal interventions — all through a clickable UI instead of raw tensor code.

Analogy: TransformerLens and nnsight are like `requests` / `curl` — powerful libraries that do the real work but require writing code for every question you ask. This tool is the Postman/browser layer on top: same underlying capability, but explorable in seconds instead of an afternoon of notebook-writing.

2. Why This Is Worth Building (and why it's not redundant)

- TransformerLens and nnsight are **libraries**, not visual tools — every inspection requires writing Python and knowing tensor shapes.
- Neuronpedia is the closest existing tool, but it's narrow: hosted SAE-feature browsing only, tied to specific pre-computed model/layer combos — not a general "point at any model and explore everything" tool.
- No widely-used tool currently gives a unified, clickable interface across residual stream + attention + neurons + SAE features + logit lens + patching + causal tracing + model comparison, for arbitrary models.
- The gap is genuinely in **UX/tooling**, not in underlying capability — which is a legitimate, scoped thing to build without needing to out-engineer the libraries themselves.
- Target users: (1) mech interp researchers who want faster exploration during the "first five minutes" of a new hypothesis, and (2) newcomers/students building intuition before they're fluent in writing raw hook code.

Honest framing: the failure mode is "worse reimplementations of TransformerLens." The success mode is "the GUI/UX layer that makes TransformerLens usable in minutes instead of hours." Keep this distinction explicit in any writeup.

3. What NOT to Try to Do (Scope Discipline)

- **Not "all types of LLMs" from day one.** Decoder-only transformers (GPT, Llama, Qwen, Mistral-style) share one architectural skeleton and cover ~90% of models anyone cares about. Encoder-decoder (T5), MoE (Mixtral), and state-space models (Mamba) have fundamentally different internals and are explicit stretch goals, not launch requirements.
 - **Not training SAEs from scratch.** Use pretrained, publicly released SAEs (Neuronpedia/EleutherAI) — training your own is a separate, GPU-heavy research project on its own.
 - **Not building all 8 features simultaneously.** Each feature ships end-to-end (backend logic → interface → visual output) before the next one starts.
 - **Not building a separate "hosted demo" codebase.** The installable package IS the deployable unit — hosting anywhere later is just running the same package, not building a second product.
-

4. High-Level Architecture (Conceptual, No Code)

Four logical layers, from bottom to top:

1. **Model Engine Layer** — TransformerLens's `HookedTransformer`, doing the actual model loading and hooked forward passes. This is the foundation everything else is built on; it is not something to reinvent.
2. **Model Adapter Layer (the core abstraction)** — A single common interface (`BaseModelAdapter`) that defines what every supported model family must be able to do: load itself, run a forward pass with full activation caching, expose its residual stream, apply the logit lens, expose attention patterns, expose neuron activations, and support activation patching. Each model family (GPT-2 style, Llama style, Qwen style) implements this interface as its own adapter. This layer is what makes the tool "generic" in a real, buildable way — adding a new model family later means writing one new adapter, not touching anything else.
3. **Core Analysis Layer** — The actual interpretability logic, each piece built once and reused across every model family via the adapter interface: residual stream visualization, attention head visualization, neuron activation ranking, logit lens computation, activation patching / causal tracing, SAE feature lookup, and side-by-side model comparison.
4. **Interface Layer** — The part a researcher actually interacts with: a local web UI (sliders for layer/head/token, prompt input, visual panels for each analysis type) that calls into the Core Analysis Layer, which calls into the Adapter Layer, which calls into the Model Engine Layer.

Key architectural decision: the whole system is packaged as a single installable Python package. A researcher installs it, points it at a model, and it launches a local web interface using their own machine's compute. There is no separate hosted backend to build or maintain — deployment anywhere (a lab VPS, Hugging Face Spaces, a personal laptop) is just running the same package in a different location.

5. Feature List, in Build Order — Full Detail

Each feature is built fully (working end to end) before starting the next. For every feature: what it shows, why it matters, exactly what chart/visual type renders it, and what controls the user gets.

1. Logit Lens

- **What it shows:** What token the model "would have predicted" if generation stopped at any intermediate layer, instead of only the final layer.
- **Why it matters:** Reveals how the model's prediction evolves and sharpens layer by layer — often the very first thing researchers check on a new model.
- **Chart type:** A **line/ribbon chart** — x-axis = layer number, y-axis = top predicted token(s) at that layer, with confidence shown as line thickness or color intensity. Alternative view: a **stacked table/heatmap** — rows = layers, columns = top-5 predicted tokens per layer, cell color = probability.
- **Controls:** layer range slider, token-position selector (which position in the prompt to inspect), toggle between "top-1 token" view and "top-5 probability" view.

2. Residual Stream Visualization

- **What it shows:** The magnitude and structure of the residual stream — the running "information highway" the model writes to at every layer.
- **Why it matters:** Shows where and how much the model is adding/modifying information at each stage; spikes often indicate important computation happening at that layer.
- **Chart type:** A **2D heatmap** — x-axis = token position, y-axis = layer, cell color = activation norm/magnitude. Optional secondary view: **PCA/dimensionality-reduction scatter plot**, showing how the residual stream vector for a token moves through a reduced 2D/3D space across layers (a "trajectory" plot with layers connected by lines).
- **Controls:** norm type selector (L2 norm, max value, variance), layer range, token range, toggle PCA trajectory view on/off.

3. Attention Head Visualization

- **What it shows:** Which tokens attend to which other tokens, for every attention head in every layer.
- **Why it matters:** The core building block for understanding what any given attention head is "doing" — copying, tracking, previous-token attention, induction patterns, etc.
- **Chart type:** A **grid of small heatmaps** (one per head), each an $N \times N$ token-by-token attention matrix, arranged in a grid of (layers $\times$ heads). Clicking any cell expands it into a **full-size annotated heatmap** with actual token labels on both axes and an overlay of attention weight as color intensity. Optional **arc diagram** view — tokens laid out in a line, curved arcs connecting

attending token to attended token, arc thickness = attention weight (often more intuitive than a matrix for short prompts).

- **Controls:** layer selector, head selector (or "show all heads" grid mode), attention weight threshold slider (hide arcs/cells below a cutoff to reduce clutter), toggle between matrix view and arc-diagram view.

4. Neuron Activation Panel

- **What it shows:** For a chosen token, which MLP neurons fired most strongly, and how a specific neuron behaves across many tokens/prompts.
- **Why it matters:** Individual neurons sometimes correspond to interpretable concepts (e.g. a "base rate of a country" neuron); this panel is how those are found.
- **Chart type:** A **horizontal bar chart** — ranked list of top-firing neurons for the selected token, bar length = activation strength. Secondary view: a **token-activation heatmap strip** — a single neuron's activation value across every token in the prompt, shown as a colored strip underneath the prompt text itself (so you literally see which words "light up" a neuron).
- **Controls:** token selector, layer selector, top-k slider (how many neurons to show), search/pin a specific neuron ID to track across different prompts.

5. Model Comparison

- **What it shows:** Any of the above views computed for two models (or two checkpoints of the same model) side by side, or as a direct difference.
- **Why it matters:** Essential for questions like "what did fine-tuning change?" or "how do two model families differ internally?"
- **Chart type:** **Split-screen dual panel** — identical chart type (heatmap, logit lens curve, etc.) rendered twice side by side, synced controls (moving the layer slider updates both). Optional **diff mode** — a single heatmap showing only the *difference* between the two models' values (diverging color scale: blue = model A higher, red = model B higher).
- **Controls:** two independent model selectors, "sync scroll/zoom" toggle, diff-mode toggle, shared prompt input (same prompt run through both).

6. Activation Patching

- **What it shows:** How much of the original ("clean") output is recovered when a single activation from the clean run is transplanted into a corrupted run.
- **Why it matters:** The core causal-intervention technique in mech interp — moves from "these two things correlate" to "this component causally matters."
- **Chart type:** A **heatmap** — x-axis = token position, y-axis = layer, cell color = amount of logit-difference recovered by patching that specific layer/position (diverging color scale, since patching can help or hurt recovery).

- **Controls:** clean-prompt input, corrupted-prompt input, metric selector (logit difference, probability, custom metric), patch-target selector (residual stream, attention output, MLP output — patch different sub-components independently), single-cell drill-down (click a cell to see the actual token predictions before/after that specific patch).

7. Causal Tracing

- **What it shows:** Activation patching swept systematically across *every* layer and position at once, aggregated into one map of "importance."
- **Why it matters:** This is the single most-cited visualization style in interpretability papers (from the original ROME/causal tracing work) — a fast way to localize where in the model a fact or behavior "lives."
- **Chart type:** Same **layer × position heatmap** as patching, but computed automatically across the full sweep rather than one cell at a time, typically shown with a smoothed/interpolated color gradient. Includes a **"top contributing components" side list** — ranked table of the highest-impact layer/position pairs, so the user doesn't have to read the heatmap by eye alone.
- **Controls:** clean/corrupted prompt pair, sweep granularity (every layer vs. every N layers, for speed), component-type filter (residual / attention / MLP), export sweep results as a downloadable table.

8. SAE Feature Explorer

- **What it shows:** A chosen layer's residual stream decomposed into sparse, more human-interpretable "features," with labels where publicly available.
- **Why it matters:** Individual neurons are often polysemantic (fire for multiple unrelated concepts); SAE features are trained specifically to be more monosemantic and interpretable.
- **Chart type:** A **ranked feature list** (bar chart, feature activation strength) per token, each entry expandable to show: the feature's human-readable label (if available from Neuronpedia/EleutherAI's public index), and a **mini activation-strip** showing which other tokens in the prompt also activate that same feature.
- **Controls:** layer selector, token selector, "search features by label" box, pin/compare multiple features side by side, link-out to the feature's full public dashboard entry if available.

9. Token Attribution (additional feature worth including)

- **What it shows:** How much each input token contributed to the final output prediction, via gradient- or attention-based attribution.
- **Why it matters:** A fast, single-glance way to see "what part of the prompt mattered" without digging through layer-by-layer detail.
- **Chart type: Highlighted text view** — the input prompt rendered as normal text, but each token's background color intensity reflects its attribution score (darker = more influential). This is often the very first, most intuitive chart a new user sees.

- **Controls:** attribution method selector (gradient-based, attention-rollout, or a simple ablation-based score), output-token selector (attribution "with respect to" which predicted token).
-

6. Full Web Dashboard Layout

How the features above come together into one cohesive interface, not nine disconnected screens:

- **Top bar (always visible):**
 - Model selector dropdown (choose model family + checkpoint)
 - Prompt input box (main text field, with a small "corrupted prompt" secondary field that appears only when patching/causal-tracing mode is active)
 - Global "Run" button, plus a session indicator showing which cached run is currently loaded
 - Export button (downloads current view as PNG/SVG, or raw underlying data as JSON/CSV)
- **Left sidebar — navigation between analysis modes:**
 - Overview / Token Attribution (default landing view — the highlighted-text chart, since it's the fastest single-glance summary)
 - Logit Lens
 - Residual Stream
 - Attention Heads
 - Neurons
 - SAE Features
 - Patching & Causal Tracing
 - Model Comparison
 - Each item switches the main panel's chart type but keeps the same loaded prompt/session, so switching views doesn't require re-running anything
- **Main panel (center, largest area):**
 - Renders whichever chart type corresponds to the selected sidebar item
 - Consistent zoom/pan controls across all chart types (since several are heatmaps of varying size)
 - A persistent mini token strip pinned along the top of the main panel showing the current prompt's tokens, so the user always has positional context no matter which chart is active
- **Right sidebar — contextual controls:**
 - Changes based on which view is active (e.g., layer/head sliders for attention, top-k slider for neurons, sweep granularity for causal tracing)
 - Always includes a small "current selection" summary (which layer/head/token is currently focused) so it's clear what the main panel is showing

- **Bottom drawer (collapsible):**
- Raw data inspector — shows the actual underlying tensor values / JSON for the current view, for users who want to copy exact numbers rather than read the chart
- Session history — list of previous runs in this session, clickable to reload without re-running the model
- **Settings/about panel (accessible from top bar):**
- Device info (confirms whether running on CPU/GPU, useful for a local-first tool)
- Supported model families list
- Link to documentation / GitHub

This layout keeps a single consistent shell (top bar + sidebar navigation + main panel) while swapping only the chart type and contextual controls per feature — so the dashboard feels like one coherent tool rather than eight separate mini-apps bolted together.

7. Model Coverage Rollout

Ship in this order, smallest and simplest first:

1. **GPT-2 small (124M)** — proof of concept, cheapest to run, most-documented in existing tooling/tutorials.
 2. **Qwen2.5 0.5B / 1.5B** — directly reuses existing familiarity from prior probe/distillation work; ties the tool to a real ongoing research thread rather than being a standalone toy.
 3. **Llama-3.2 1B** — proves the adapter abstraction genuinely generalizes across model families, not just within one.
 4. **Stop here for v1.** Larger models, MoE architectures, and non-decoder-only architectures are explicitly deferred to a future version.
-

8. Compute & Deployment Strategy

- **No dedicated infrastructure required to build or ship this.** Because it's a local-first package, compute is whatever machine the user runs it on.
- **Development:** free-tier Colab/Kaggle GPU sessions are sufficient for building and testing against the small models in scope.
- **Small models are CPU-viable too:** GPT-2 small and Qwen0.5B run acceptably even without a GPU for most features; only causal-tracing sweeps (many repeated forward passes) get noticeably slower on CPU, and remain usable at this model scale.
- **Distribution:** publish as an installable package (initially installable directly from source/GitHub, later formally published once stable).

- **Optional hosted demo:** once the package is stable, a minimal hosted instance (e.g. on Hugging Face Spaces using the smallest model) can serve as a "try before install" entry point — this is just the same package running elsewhere, not a second thing to build or maintain.
 - **Lab/VPS use:** any lab wanting a persistent shared instance can run the identical package on their own server — again, no separate version needed.
-

9. Realistic Build Timeline

Approached as a sequence of fully-working slices, not a simultaneous mega-build:

- **Phase 0 — Setup:** environment, repo, package skeleton, dependency installation, first successful model load and forward pass.
- **Phase 1 — Foundation:** the model adapter abstraction defined, with a single working adapter (GPT-2 family) implemented against it.
- **Phase 2 — First full vertical slice:** logit lens feature working completely, from model internals through to a visible, interactive result. This is the proof that the whole stack (engine → adapter → analysis → interface) works end to end.
- **Phase 3 — Feature expansion:** residual stream viz, attention heads, and neuron activations added one at a time, each following the same proven pattern from Phase 2.
- **Phase 4 — Model family expansion:** Llama and Qwen adapters added, confirming the existing UI and features work unmodified against new model families — the real validation of the adapter design.
- **Phase 5 — Advanced features:** activation patching, causal tracing, and model comparison, which are more conceptually involved and best tackled once the simpler features and adapter pattern are proven solid.
- **Phase 6 — SAE integration:** pretrained SAE loading and feature exploration, treated as the most optional/stretch feature given its dependency on external pretrained artifacts.
- **Phase 7 — Polish & distribution:** documentation, example notebooks, packaging for distribution, and (optionally) a hosted demo instance.

Realistic total effort: several months of consistent part-time work, given this runs alongside other commitments — sequencing matters more than speed here, since each phase depends on the previous one being solid.

10. Positioning & Differentiation (for README / writeups)

- **Not competing with TransformerLens or nnsight** — built directly on top of one of them, using their hook/caching primitives as the foundation.

- **Not attempting to be Neuronpedia** — broader in scope (full interactive debugging across multiple analysis types) rather than SAE-features-only.
 - **The pitch, in one line:** "The interactive, visual layer that makes existing mechanistic interpretability libraries usable in minutes instead of an afternoon of custom notebook code."
 - **Audience framing:** useful both to experienced researchers (fast hypothesis-checking) and newcomers to the field (building intuition before writing raw hook code).
-

11. Why This Fits the Broader Trajectory

- Directly extends existing hands-on work with hallucination probes and knowledge distillation on Qwen2.5 models — this isn't a disconnected side project, it's tooling for research already underway.
- A working, documented interpretability tool is a distinctive, citable artifact for graduate school applications in AI/ML, particularly for programs with any interpretability or safety research focus.
- Demonstrates both the research-adjacent skill (understanding transformer internals deeply enough to build tooling for them) and the engineering skill (full-stack, packaging, deployment) already part of an existing skill set.